



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.3 Latency hiding techniques

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

1. Introduction
2. The protocol stack
3. Impact on programming of on-line games (4 sessions)
 - Summary and general aspects
 - Latency, jitter, and reliability
 - **Latency hiding techniques**
 - Scalability
4. Other stories

Remember from last class



- There are different sources of latency at the host equipment and also the network.
- Network latency is the most unpredictable, and packets can be disordered and lost.
- TCP is powerful but slow, games generally use UDP + custom reliability based on similar principles than TCP
- To save bandwidth, we have several tricks for compressing the data to send

- There are different sources of latency at the host equipment and also the network.
- Network latency is the most unpredictable, and packets can be disordered and lost.
- TCP is powerful but slow, games generally use UDP + custom reliability based on similar principles than TCP
- To save bandwidth, we have several tricks for compressing the data to send
- **What do we do with the latency that we cannot avoid?**



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

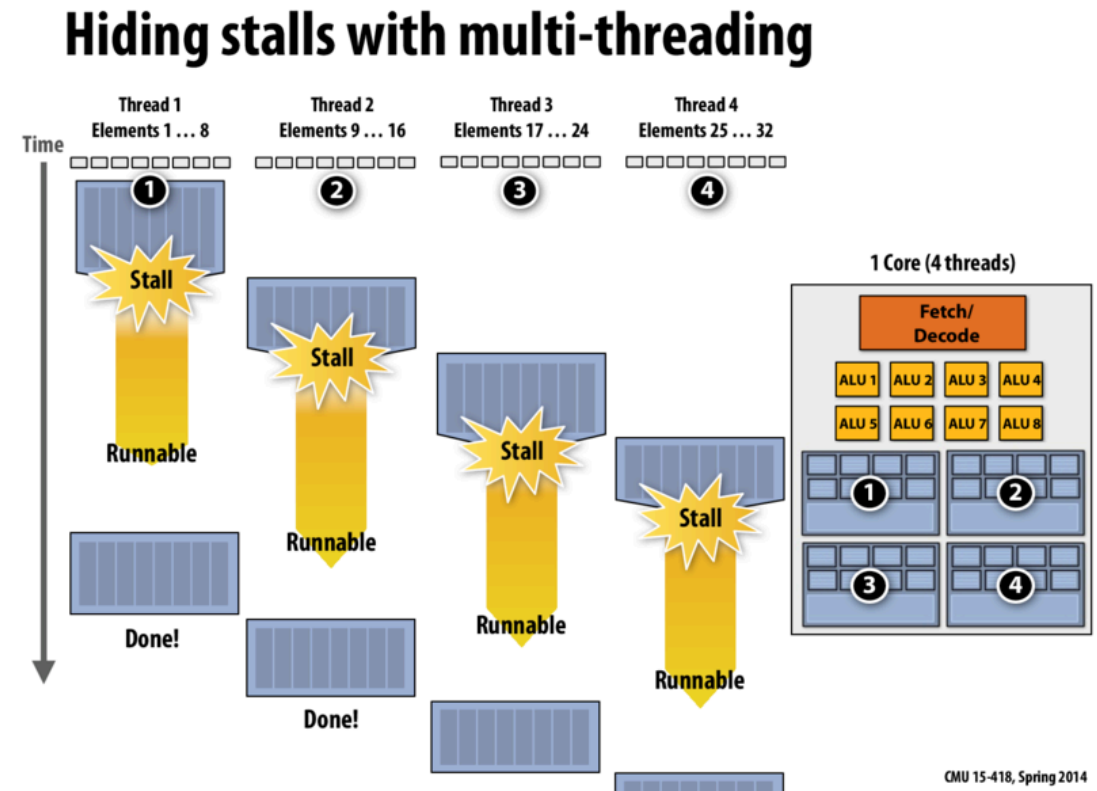
Centre de la Imatge i la Tecnologia Multimèdia

Introduction to Latency Hiding

- Computer architectures in general and parallel architectures in particular have always looked for ways to hide latency of their operations
- Basically, you try to perform useful work while waiting for a delayed operation to finish. If you do not find work to do, the processor will stall (stop for a few micro/milliseconds)

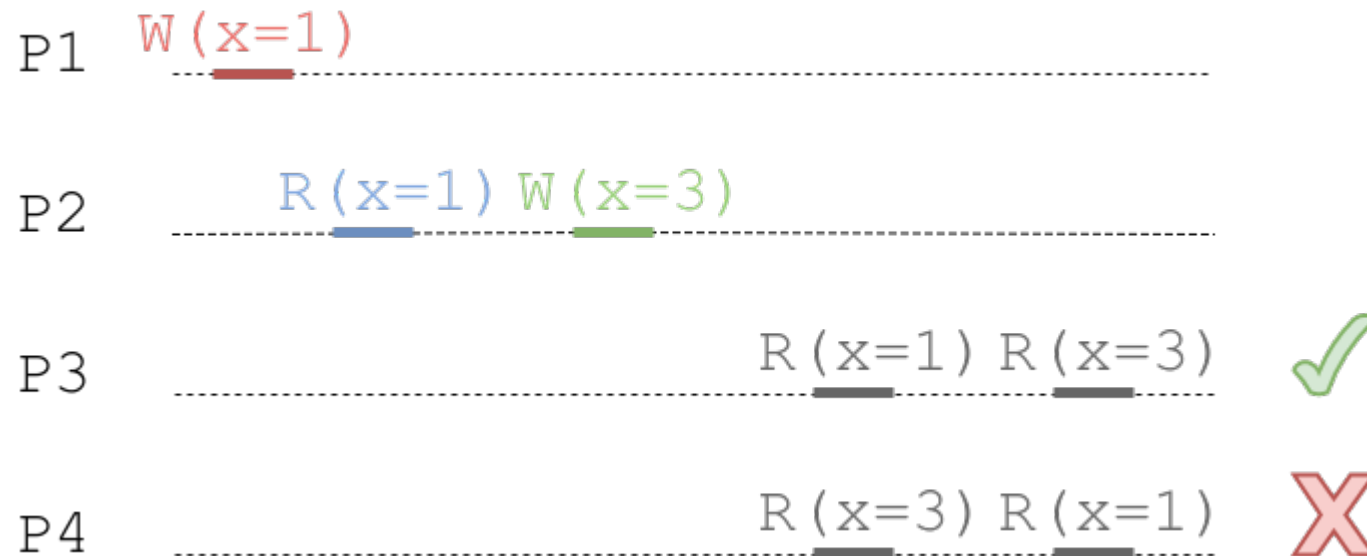
Latency hiding

- Computer architectures in general and parallel architectures in particular have always looked for ways to hide latency of their operations
 - Example: multithreading



Latency hiding

- Computer architectures in general and parallel architectures in particular have always looked for ways to hide latency of their operations
 - Example: relaxed consistency and speculative execution



- On-line games are distributed systems with very high and variable latencies that will disturb the quality of experience. The designer has to make a large effort to hide its detrimental effects.
- Most techniques basically play with the fact that the user may tolerate 100 ms (action games) and use that time in its favor.
- Also, the physics and human behavior play a role. We can guess what could happen and be right most of the times.



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

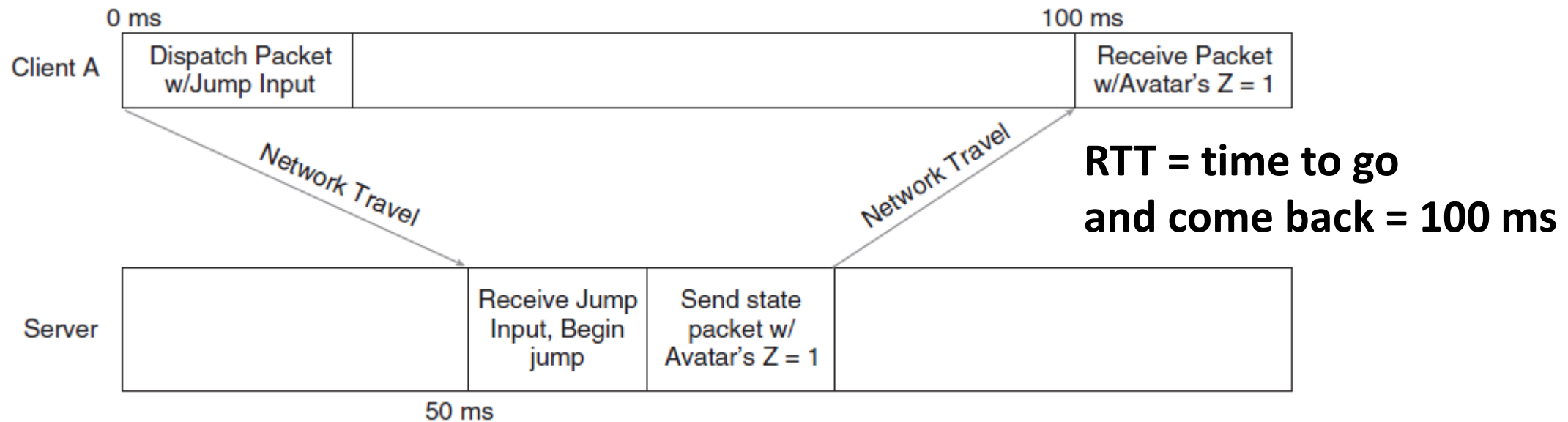
Latency hiding in games: Beyond the dumb client

Lab Session 9

Jitter and Latency Handling

The dumb terminal client

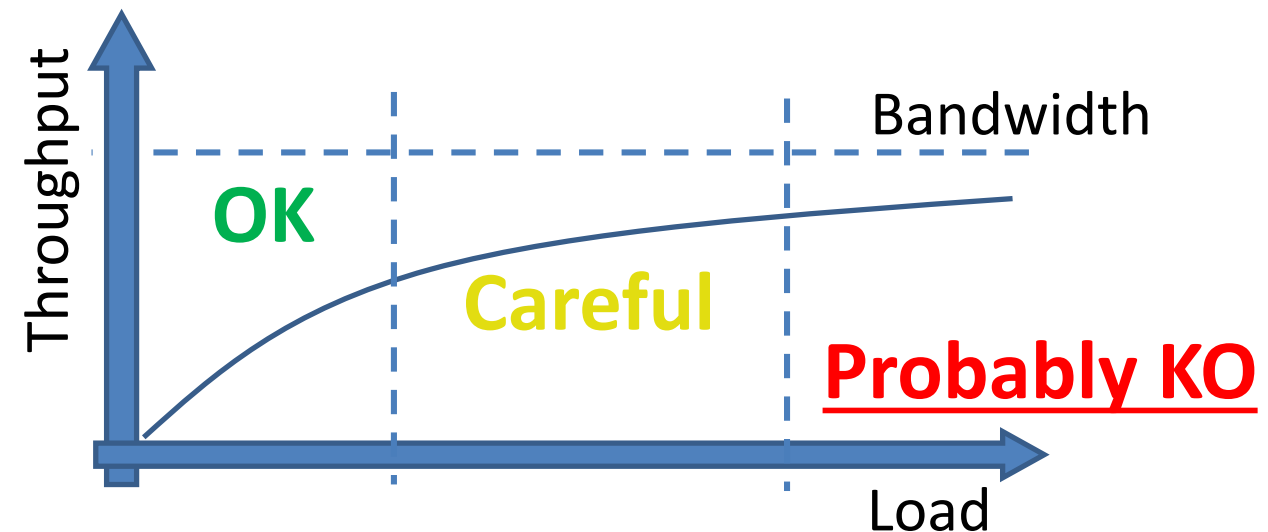
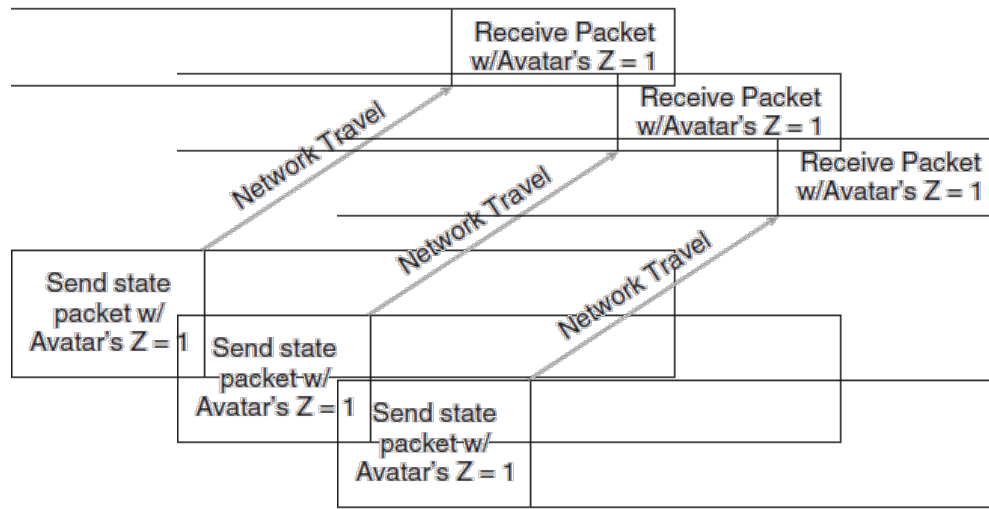
- First games just sent inputs and received state



- The user is always “one network travel behind” the server
- Always correct, because you always wait for servers state

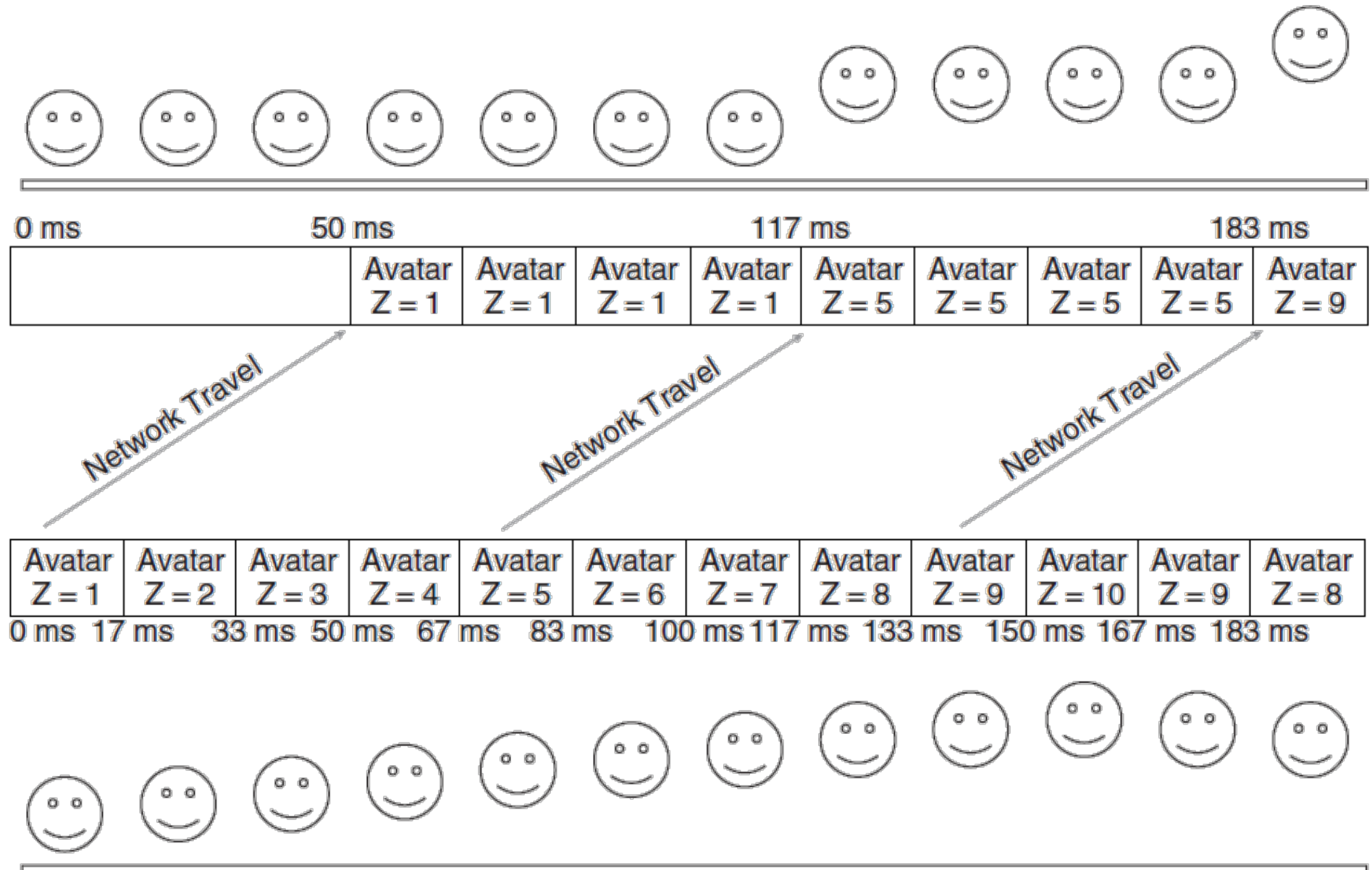
The dumb terminal client

- Dumb terminals have several problems
 - If network latency increases, the game immediately lags.
 - You can only refresh the client state as many times as the server communicates. This is not friendly with bandwidth.



The dumb terminal client

The effect
of a lack of
updates is:



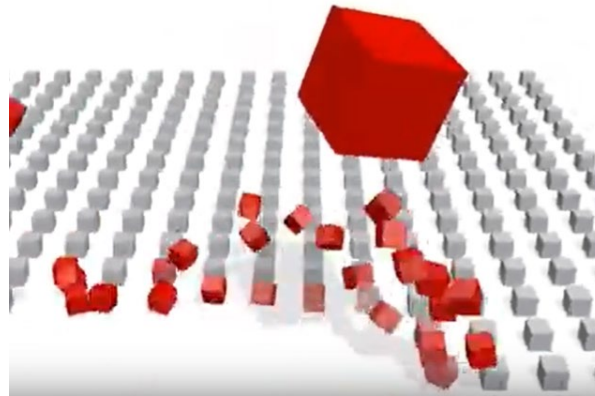
Exercise



CITM

Estimate the amount of bandwidth needed in the “cube simulator” by sending 60 positions per second, assuming that we have a dumb terminal and a naïve implementation of replication of world state. There are 900 cubes and their implementation is:

```
struct CubeState
{
    bool interacting;
    vec3f position;
    vec3f linear_velocity;
    quat4f orientation;
};
```



Exercise



Estimate the amount of bandwidth needed in the “cube simulator” by sending 60 positions per second, assuming that we have a dumb terminal and a naïve implementation of replication of world state. There are 900 cubes.

$$B = 900 \frac{\text{cubes}}{\text{frame}} * 321 \frac{\text{bits}}{\text{cube}} * 60 \frac{\text{frames}}{\text{second}} = 17.334 \text{ Mb/s}$$

Naïve replication of world state (all cubes)

Cube struct

Dumb terminal



Exercise



CITM

Estimate the amount of bandwidth needed in the “cube simulator” by sending 60 positions per second, assuming that we have a dumb terminal and a naïve replication of world state. There are 900

Not affordable.
How to improve?

$$B = 900 \frac{\text{cubes}}{\text{frame}} * 321 \frac{\text{bits}}{\text{cube}} * 60 \frac{\text{frames}}{\text{second}} = 17.334 \text{ Mb/s}$$

World replication as
Object world state
(all cubes)

Snapshot
compression

???
Dumb
terminal



Exercise



CITM

Estimate the amount of bandwidth needed in the “cube simulator” by sending 60 positions per second, assuming that we have a dumb terminal and a naïve replication of world state. There are 900

Not affordable.
How to improve?

$$B = 900 \frac{\text{cubes}}{\text{frame}} * 321 \frac{\text{bits}}{\text{cube}} * 60 \frac{\text{frames}}{\text{second}} = 17.334 \text{ Mb/s}$$

World state deltas
Object state deltas
(Lesson 3.1)

Snapshot
compression

???



- Dumb terminals have several problems
 - If network latency increases, the game immediately lags.
 - You can only refresh the client state as many times as the server communicates. This is not friendly with bandwidth.
 - This means that even if we render at 60 FPS, we only see at the speed at which server sends updates. Lack of fluidity can be frustrating for fine-grained moves (e.g. aiming)
 - **We need to ways to do reduce the number of updates and do useful work while waiting for these updates.**



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Latency hiding in games: Interpolation

Lab Session 9

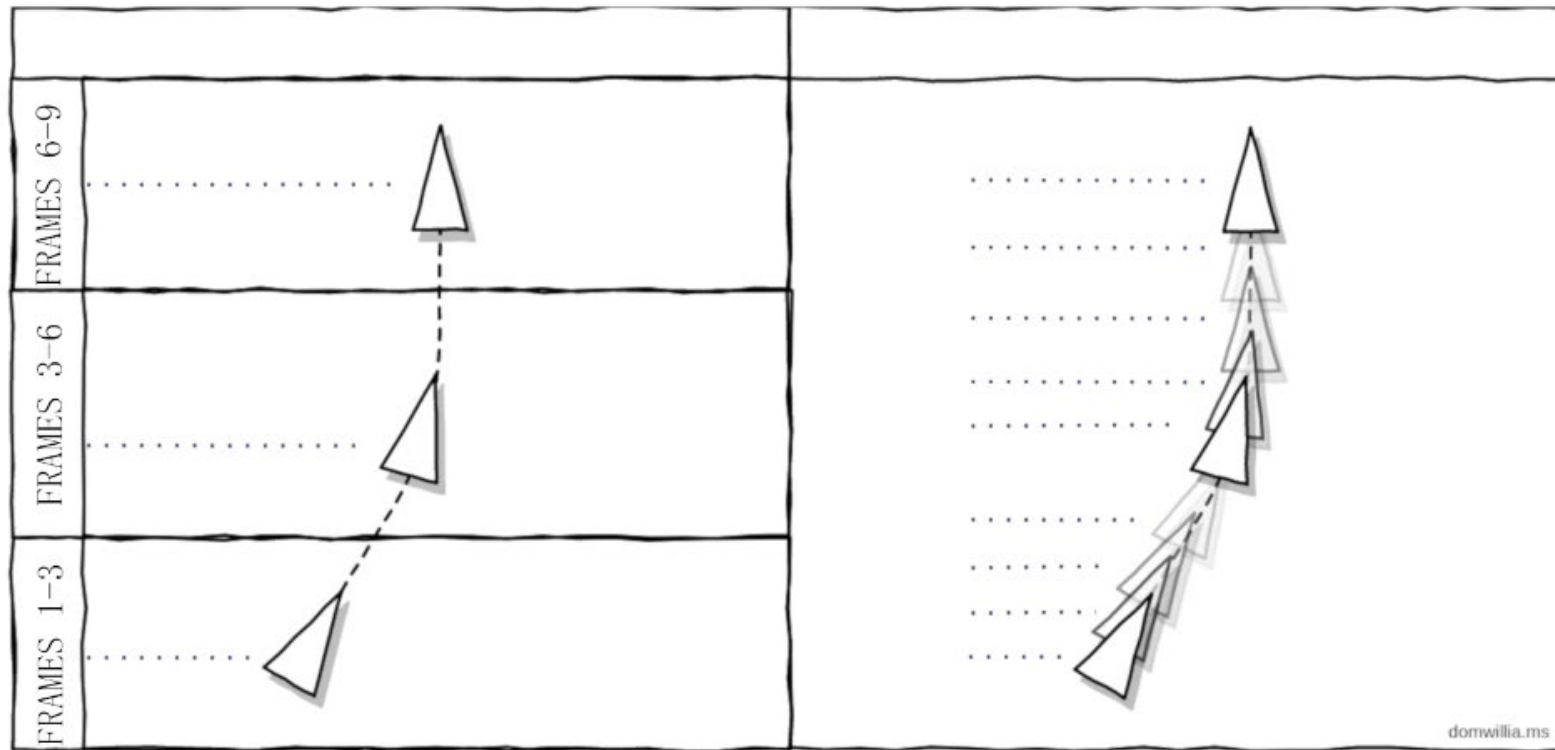
Jitter and Latency Handling

- **In snapshot interpolation**, the interpolation process allows to give the user the 60 FPS fluidity without having to send 60 positions per second.
- Basically, clients do not immediately apply the new state given by the server, but interpolate slowly between their current state and the new state.
- Interpolation is made with a local perception filter, it can be very simple or a bit more sophisticated.
- The state seen by the client is always correct, because the interpolation occurs between two correct states.

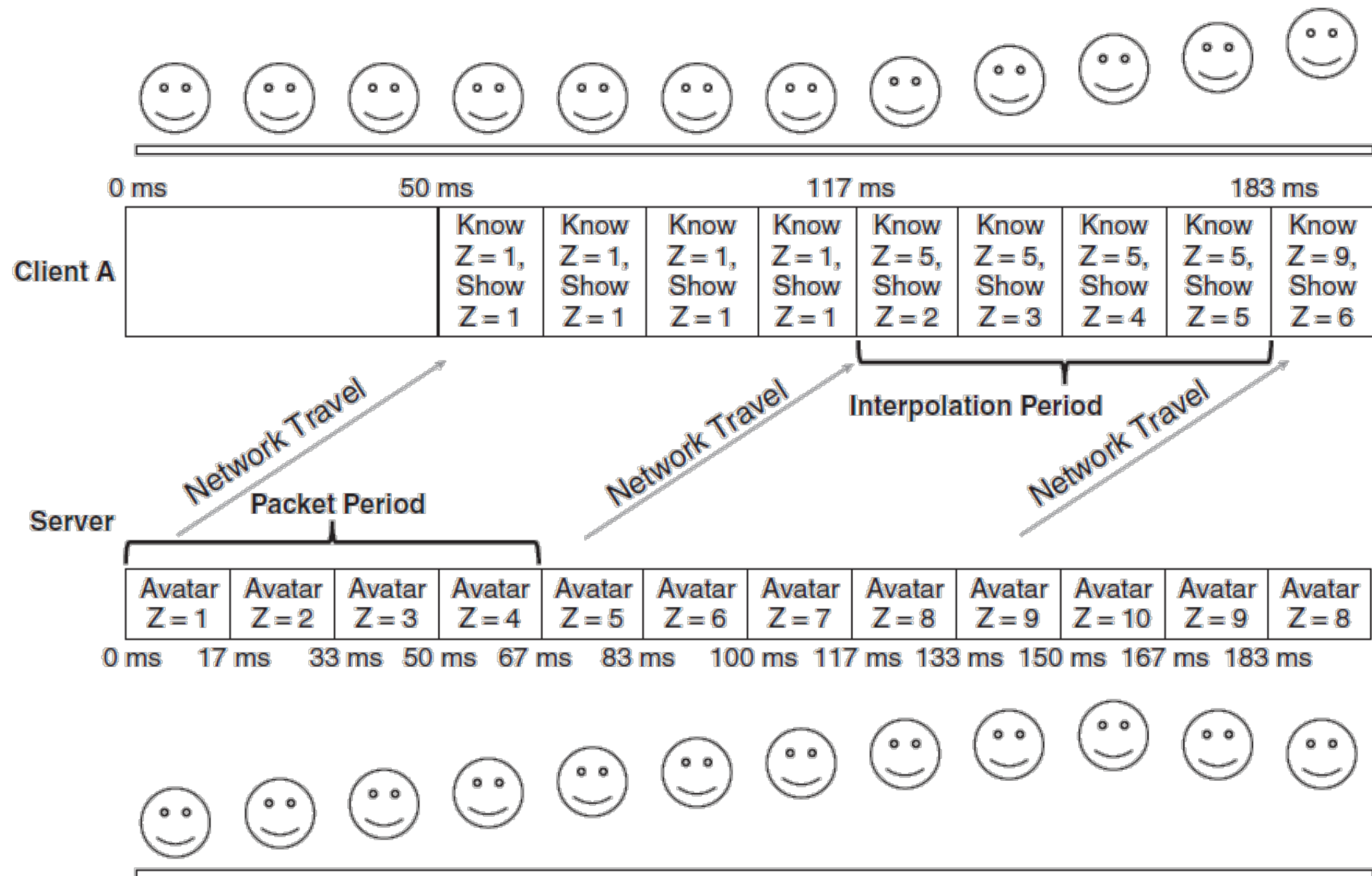
Client Side Interpolation

Solution: interpolate position between 2 known points in time

From the lab sessions



Client Side Interpolation



- There are two periods:
 - The packet period (PP): time between sent packets
 - The interpolation period (IP): duration of the interpolation
- Now the client is behind the server by $IP + RTT/2$
- IP has to be chosen carefully to not delay the client too much and still have a fluid experience
 - If $IP < PP$, then there will be a small jump when updating from non-interpolated state to the new state.
 - If $IP > PP$, the client may be delayed too much.
 - Ideally, $IP \approx PP$.

Videos (snapshot interpolation)

- <https://youtu.be/UwAw3Mtl1cM> – 60 pps, jitter
- <https://youtu.be/SPm4-KDW-Yk> – 10 pps, no interpolation
- <https://youtu.be/DjQmYrKd92U> – 10 pps, interpolation



Which is the relative bandwidth requirement of the interpolated version?



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Latency hiding in games: Prediction

Lab Session 9

Jitter and Latency Handling

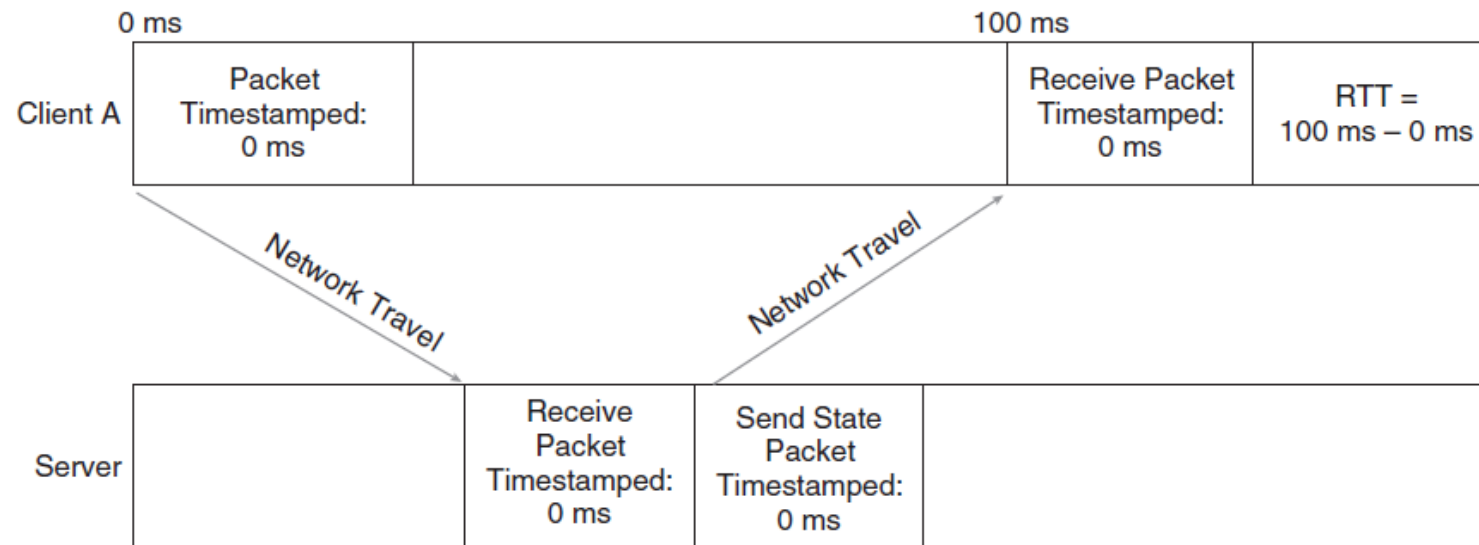
- The problem of interpolation is that it adds delay. We keep seeing old state!
- To see newer state we need to extrapolate. This is done using **client side prediction**.
- **In state synchronization**, we predict the next states during $RTT/2$ and, when the actual state is received from the server, we correct if needed.
- This is an algorithm that may be wrong at times. Unlike interpolation or dumb terminal, which are conservative, client-side prediction is optimistic.

- Aspects of client side prediction
 - RTT Estimation
 - Dead Reckoning
 - Client Move Prediction and Replay
 - Tricks to further hide latency

- Aspects of client side prediction
 - **RTT Estimation**
 - Dead Reckoning
 - Client Move Prediction and Replay
 - Tricks to further hide latency

RTT Estimation for CSP

- First step is to estimate the RTT to understand how much of state needs to be predicted every time.
- Clocks in distributed systems are not synchronized, so we need an alternative.
- Timestamping is generally used.



- First step is to estimate the RTT to understand how much of state needs to be predicted every time.
- Clocks in distributed systems are not synchronized, so we need an alternative.
- Timestamping is generally used.
 - Send timestamped packet.
 - Receiver sends the exact same packet back
 - Original sender receives the packet and uses the timestamp to calculate the RTT.

RTT Estimation for CSP

- First step is to estimate the RTT to understand how much of state needs to be predicted every time.
- Clocks in distributed systems are not synchronized, so we need an alternative.
- Timestamping is generally used.
 - Send timestamped packet.
 - Receiver sends the exact same packet back.
 - Original sender receives the packet and calculates the RTT.

REMEMBER: $RTT/2$ is an approximation of how old the received data is. This is because networks are not symmetric, and latency $A \rightarrow B$ is generally not the same than latency $B \rightarrow A$ (actually, it can differ a lot)

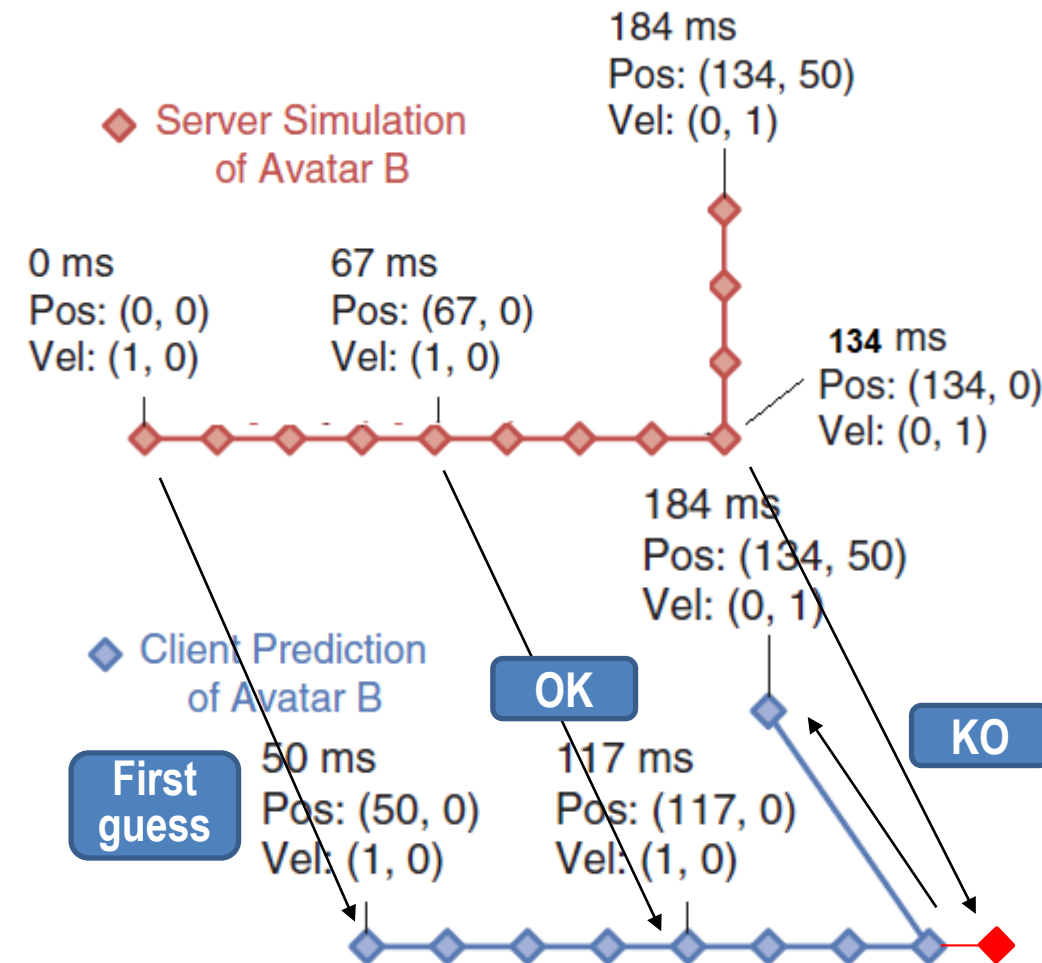
- Aspects of client side prediction
 - RTT Estimation
 - **Dead Reckoning**
 - Client Move Prediction and Replay
 - Tricks to further hide latency

- Most movements in the game are deterministic and can be simulated equally well in the client than in the server
 - Bullets through the air
 - Balls bouncing off walls
- However, movements made by the players are not deterministic and we need to make educated guesses
- If we were wrong in the guess, we need to correct.

- **Dead Reckoning** is the process of predicting an entity's behavior based on inertia: he will keep doing whatever is currently doing.
 - Example: a running player will keep running.
- Thus, objects controlled by the player will be simulated as if there was no change of input.
- If there is actually a change of input, we will know later. In the meanwhile, the client simulation will diverge from the actual state.

Dead Reckoning

- Example:
 - $RTT = 100\text{ ms} \rightarrow RTT/2 = 50\text{ ms}$
 - 60 FPS
 - First Pos received by client = (0, 0)
 - First Vel received by client = (1, 0)
 - First Pos guessed at $RTT/2 = (50, 0)$
 - Second Pos extrapolated OK
 - Jump at 134 ms, this is misguessed
 - Client needs to correct



How do we remedy the situation?

- **Instant state update:** take the new position immediately and start simulating from there.
- **Interpolation:** use the current wrong position and the actual one to interpolate and minimize the error gracefully. Cubic spline interpolation can be used here.
- **Second-order state adjustment:** depending on the change of state (static to moving object) we may want to make second-order calculations to smoothen changes.
- **Combinations of the above** depending on distance.

Videos (state synchronization)



CITM

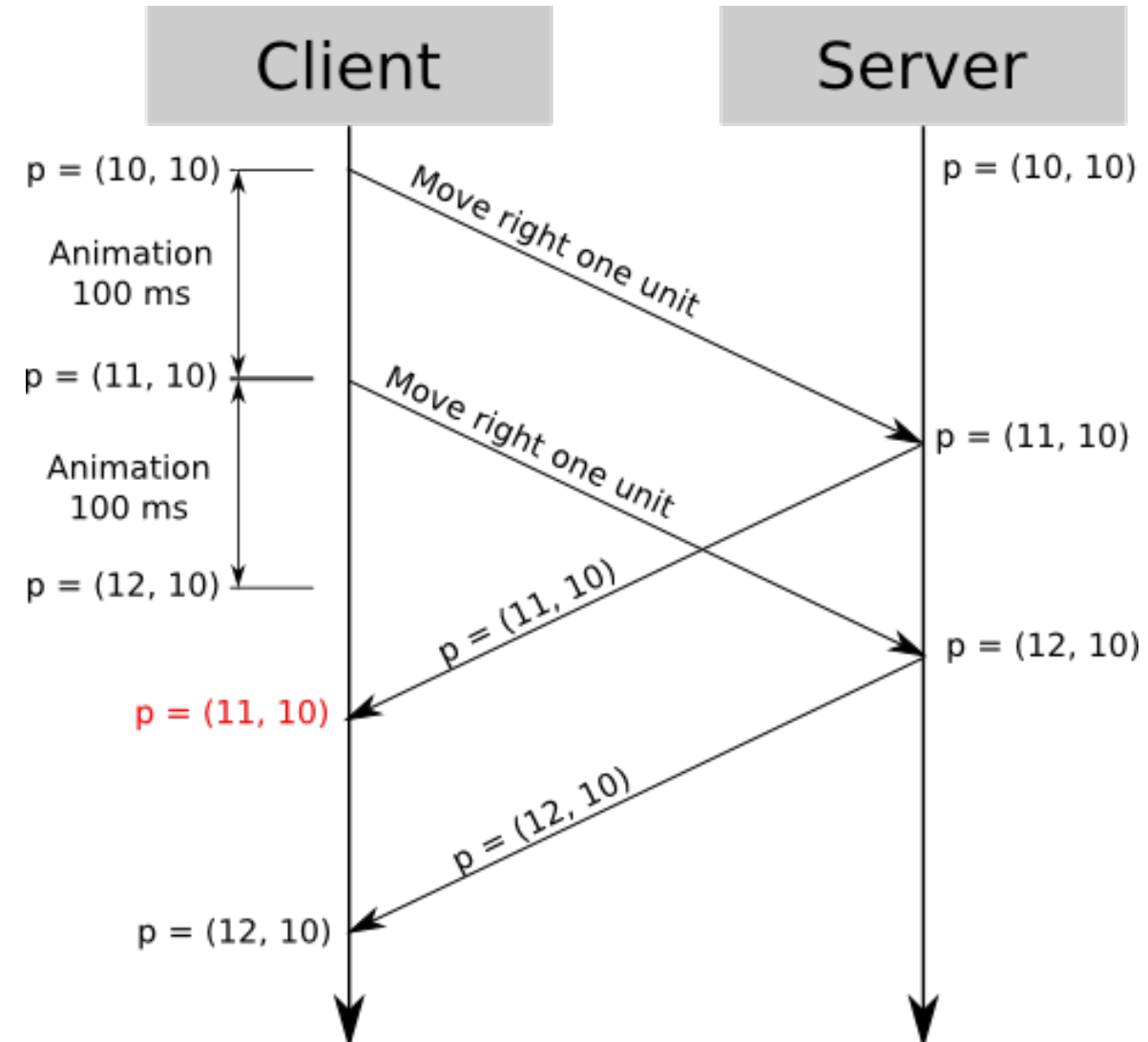
- <https://youtu.be/egGmccwILdU> - OK
- <https://youtu.be/4ESA5u3bMwY> - Smoothing
- <https://youtu.be/vz0N77rn4VQ> - Adaptive smoothing

- Aspects of client side prediction
 - RTT Estimation
 - Dead Reckoning
 - **Client Move Prediction and Replay**
 - Tricks to further hide latency

- Dead reckoning hides latency of remote users actions, but not for the local player.
- Local player presses a button and still has to wait a full RTT to start using dead reckoning.
- **Client Move Prediction and Replay** basically allows the local client to use its own commands “without latency”.
 - Prediction: assume that my command will be executed in the server as it has been in my local machine.
 - Replay: corrects potential divergences from the prediction above.

Client Move Prediction and Replay

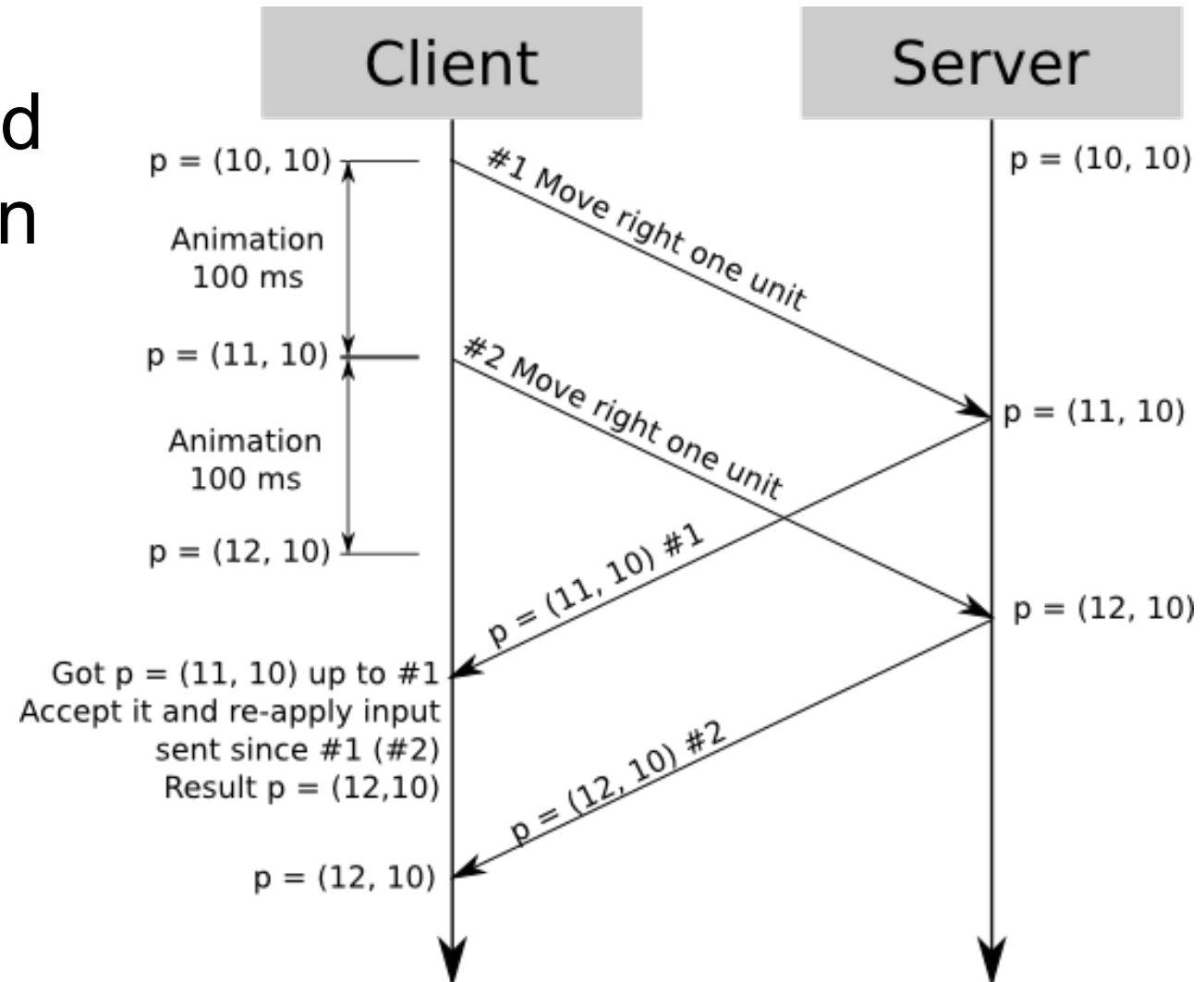
- Imagine there is lag, I would be executing many things in local and the server would take some time to react.
- Without replay, this could happen:
 - $10 \rightarrow 11 \rightarrow 12 \rightarrow 11 \rightarrow 12$



- The catch here is that you cannot correct divergences in the same way that you do with dead reckoning, interpolating, because it does not feel right. Your next state should be re-calculated instead of interpolated.
- We use **moves – input state tied to a timestamp**
 - Client A executes his actions for $T = 100$
 - Client A sends the actions for $T = 100$ to Server
 - Client A stores the actions for $T = 100$ for a while
 - As the server later provides the state for $T = 100$, Client A takes the stored actions and replays – resolving collisions.

Client Move Prediction and Replay

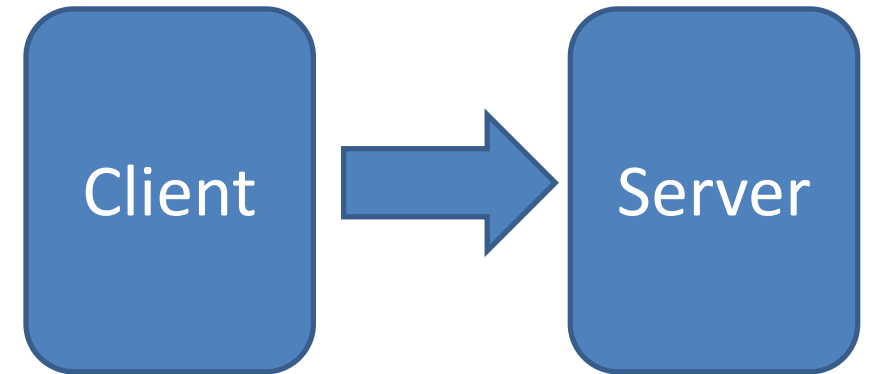
- Imagine there is lag, I would be executing many things in local and the server would take some time to react.
- With replay, this happens:
 - $10 \rightarrow 11 \rightarrow 12 \rightarrow \underline{12} \rightarrow 12$



Client Move Prediction and Replay

```
void NetworkManagerClient::SendInputPacket()
{
    const MoveList& moveList = InputManager::sInstance->GetMoveList();
    if(moveList.HasMoves())    If moves are ready to be sent...
    {

        OutputMemoryBitStream inputPacket;
        inputPacket.Write(kInputCC);
        mDeliveryNotificationManager.WriteState(inputPacket);
        //write the 3 latest moves for added reliability!
        int moveCount = moveList.GetMoveCount();
        int firstMoveIndex = moveCount - 3;
        if(firstMoveIndex < 3)    Send latest three
        {                        to add reliability
            firstMoveIndex = 0;
        }
        auto move = moveList.begin() + firstMoveIndex;
        inputPacket.Write(moveCount - firstMoveIndex, 2);
        for(; firstMoveIndex < moveCount; ++firstMoveIndex, ++move)
        {
            move->Write(inputPacket);
        }
        SendPacket(inputPacket, mServerAddress);
    }
}
```



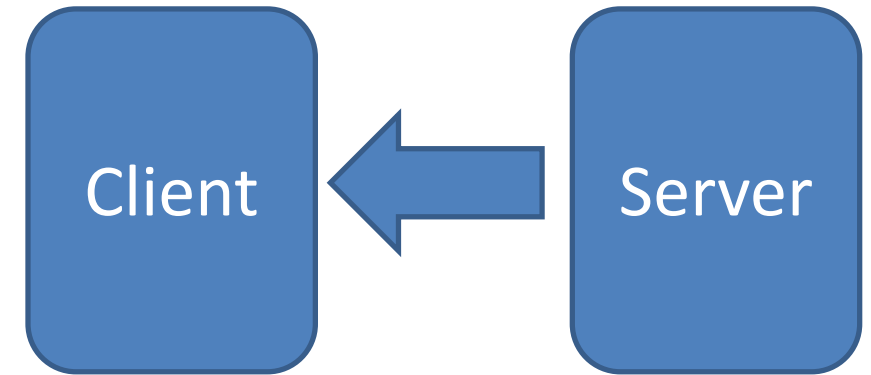
Client sends packets and keeps the moves so that they can be used for move replay after receiving server state.

Write the contents of the move into the packet that is sent.

Client Move Prediction and Replay

```
void
NetworkManagerClient::ReadLastMoveProcessedOnServerTimestamp(
    InputMemoryBitStream& inInputStream)
{
    bool isTimestampDirty;
    inInputStream.Read(isTimestampDirty);
    if(isTimestampDirty)
    {
        inPacketBuffer.Read(mLastMoveProcessedByServerTimestamp);
        mLastRoundTripTime = Timing::sInstance.GetFrameStartTime()
            - mLastMoveProcessedByServerTimestamp;
        InputManager::sInstance->GetMoveList().
            RemovedProcessedMoves(mLastMoveProcessedByServerTimestamp);
    }
}
```

We first read the timestamp
that the server has last processed



When the client receives a state packet,
it uses *RemovedProcessedMoves* to
remove already read moves.

Once finished, the client move list contains
only moves that the server has not yet seen,
and should thus be replayed.

```
void MoveList::RemovedProcessedMoves(
    float inLastMoveProcessedOnServerTimestamp)
{
    while(!mMoves.empty() &&
        mMoves.front().GetTimestamp() <=
            inLastMoveProcessedOnServerTimestamp)
    {
        mMoves.pop_front();
    }
}
```

If timestamp of the
moves pending to
confirm are
previous to the
expected
timestamp, they
can be removed.

- Aspects of client side prediction
 - RTT Estimation
 - Dead Reckoning
 - Client Move Prediction and Replay
 - **Tricks to further hide latency**

Tricks to further hide latency

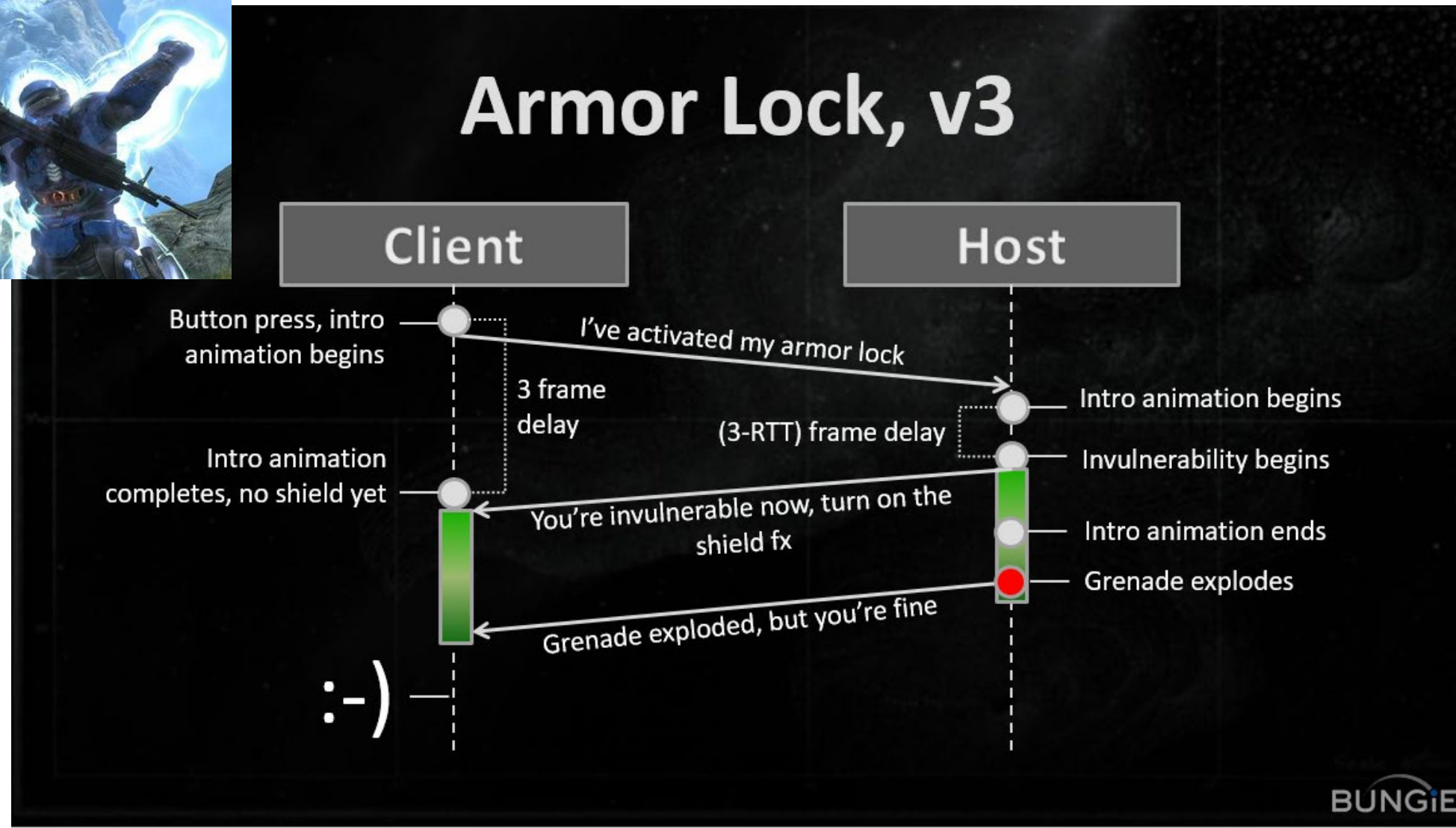
- Almost all vide game actions have **tells**, visual/audio cues that indicate that something is happening.
 - Flashes preceding plasma blasts
 - Hand waving before launching a spell
- Tells provide instantaneous feedback (the user knows that the spell or shot is being prepared)
- Tells have an approximate duration of RTT so that the game can successfully simulate the correct outcome



Armor Lock in HALO as an example

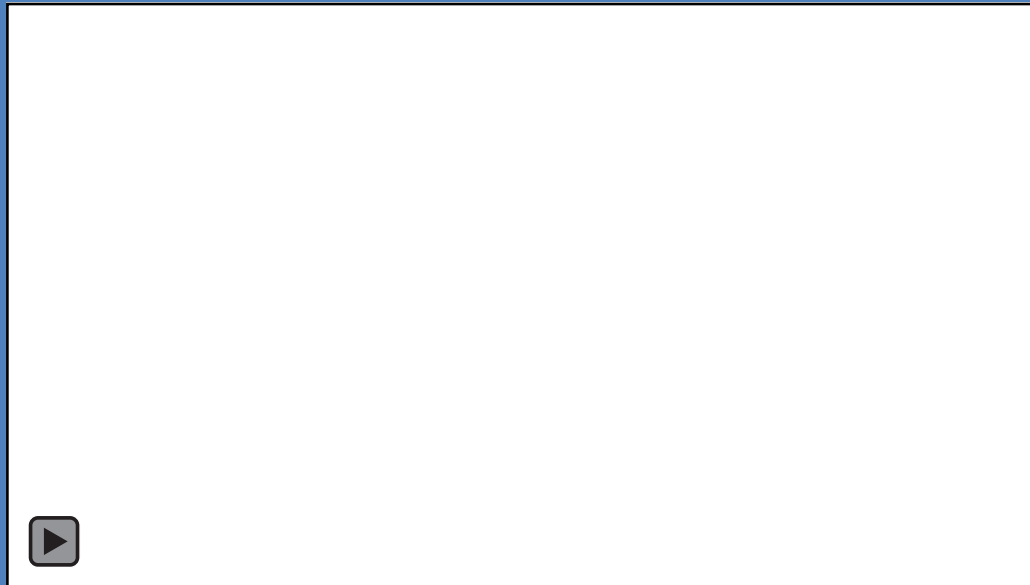


Armor Lock, v3



- Aspects of client side prediction

- Client side prediction is also called rollback netcode and it's used in fighting games
 -
 -
 -
 -



<https://words.infil.net/w02-netcode-p4.html>



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Latency hiding in games: Updating the server

Lab Session 9

Jitter and Latency Handling

Server Side Rewind (also Lag Compensation)



CITM

- Client side techniques allow to provide a fairly responsive experience even with moderate latency.
- However, there is one case that is hard to handle: long-range, instant-hit weapons. You want sniper rifles to kill the remote user instantly, and not miss if it is clear that you were targeting the remote user.
- To do this, Valve introduced **Server Side Rewind** in games such as Counter-Strike. SSR mainly rewinds server state to match that of the client that has shot
 - This in the end can be used for any kind of shot

Server Side Rewind (also Lag Compensation)

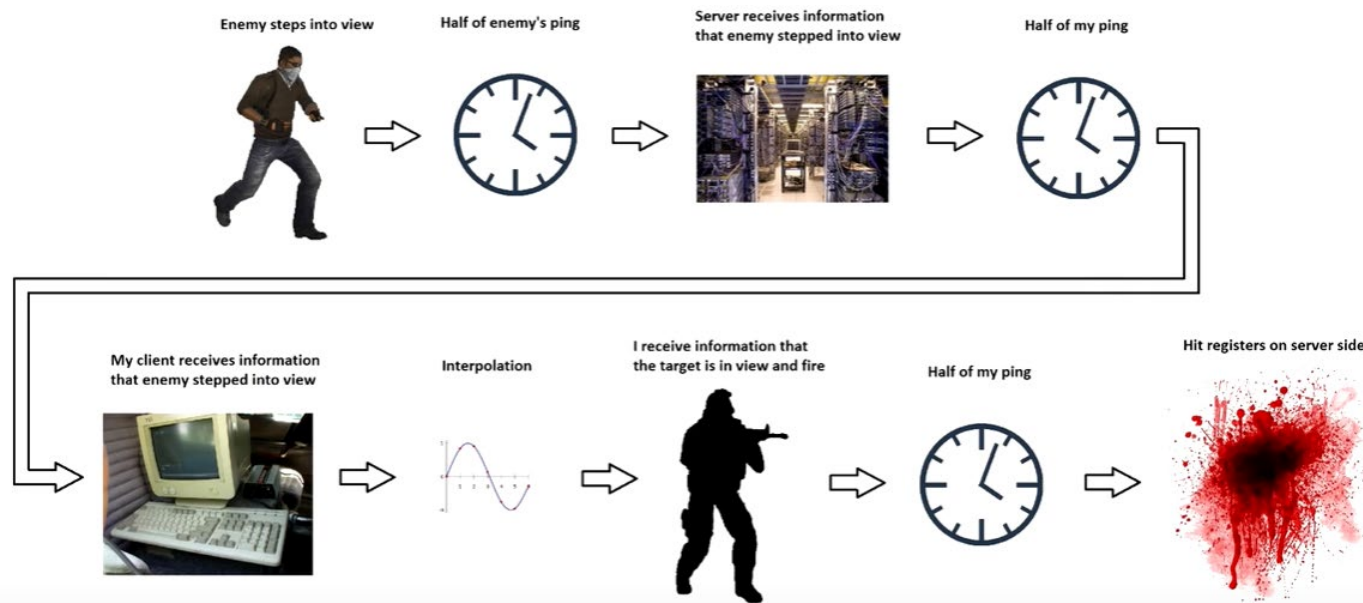


CITM

- **Do not use dead reckoning for remote players.**
Instead, we will use interpolation. This way, the server knows what the clients are seeing all the time.
- **Use client move prediction and replay** to increase responsiveness.
- **Record the client's view and send it to the server** to further augment the server's control on who sees what
- **At the server, store poses of relevant objects for a few frames** to calculate exactly the position of everyone at the moment the shot was fired.

Server Side Rewind (also Lag Compensation)

- <https://www.youtube.com/watch?v=6EwaW2iz4iA>





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.3 Latency hiding techniques

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs